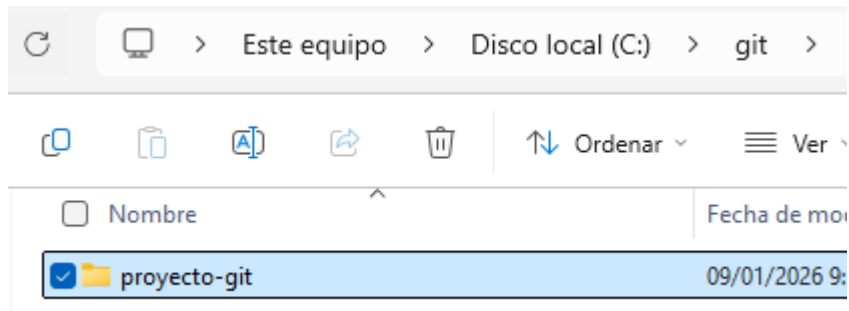


Ejercicio 1: Inicialización y primeros commits:

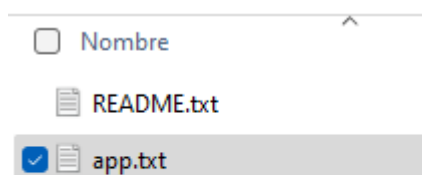
Primero nos dirigimos a esta ruta y creamos una carpeta sobre la que vamos a trabajar



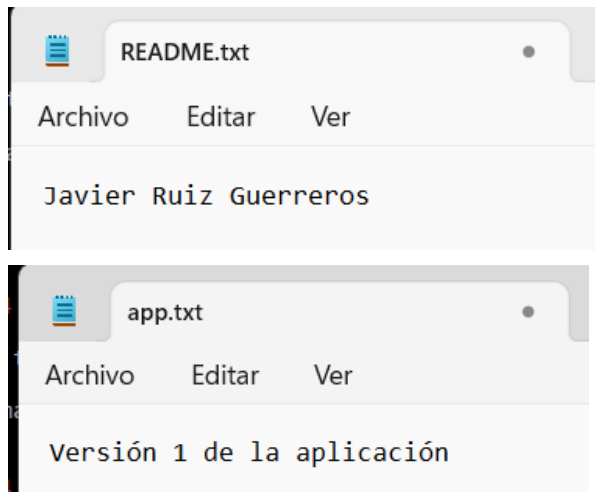
Después entramos a git y cambiamos a la carpeta creada, después iniciamos

```
Alumno04@PC13 MINGW64 ~  
$ cd c:  
  
Alumno04@PC13 MINGW64 /c  
$ cd git  
  
Alumno04@PC13 MINGW64 /c/git (main)  
$ cd proyecto-git  
  
Alumno04@PC13 MINGW64 /c/git/proyecto-git (main)  
$ git init  
Initialized empty Git repository in C:/git/proyecto-git/.git/
```

Creamos los .txt



Con este contenido



Hacemos un git status para comprobar el estado, un git add . para mandarlo al staging area y un git commit con un nombre

```
Alumno04@PC13 MINGW64 /c/git/proyecto-git (master)
$ git status
On branch master

No commits yet

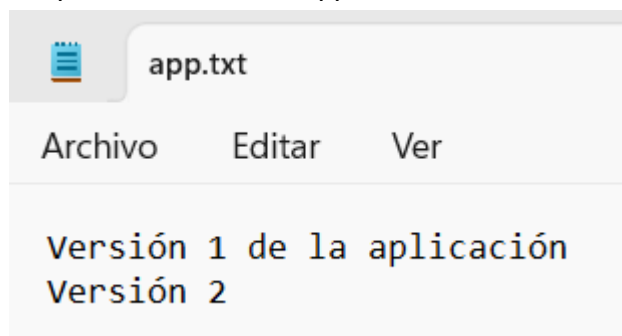
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        README.txt
        app.txt

nothing added to commit but untracked files present (use "git add" to track)

Alumno04@PC13 MINGW64 /c/git/proyecto-git (master)
$ git add .

Alumno04@PC13 MINGW64 /c/git/proyecto-git (master)
$ git commit -m "Primer commit de la aplicación"
[master (root-commit) 3cd9aa7] Primer commit de la aplicación
 2 files changed, 2 insertions(+)
 create mode 100644 README.txt
 create mode 100644 app.txt
```

Después modificamos app.txt



Y realizamos un segundo commit

```
Alumno04@PC13 MINGW64 /c/git/proyecto-git (master)
$ git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   app.txt

no changes added to commit (use "git add" and/or "git commit -a")

Alumno04@PC13 MINGW64 /c/git/proyecto-git (master)
$ git add .

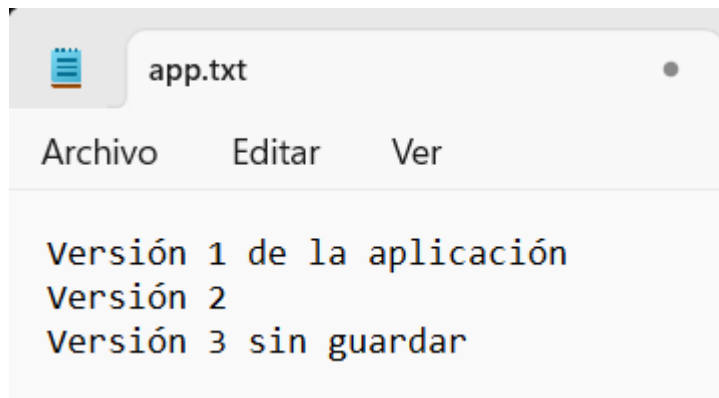
Alumno04@PC13 MINGW64 /c/git/proyecto-git (master)
$ git commit -m "Segundo commit de la aplicación"
[master e4c3e43] Segundo commit de la aplicación
 1 file changed, 2 insertions(+), 1 deletion(-)
```

Ejercicio 2 – Historial y deshacer cambios:

Para ver el historial de commits realizamos un git tree

```
Alumno04@PC13 MINGW64 /c/git/proyecto-git (master)
$ git tree
* e4c3e43d58187ab0e5af3971d82bf7183e9d675b (HEAD -> master) Segundo commit de la aplicación
* 3cd9aa70f9880f3705f582f62f11c83ca09137a1 Primer commit de la aplicación
```

Ahora modificamos de nuevo app.txt



Y hacemos un status

```
Alumno04@PC13 MINGW64 /c/git/proyecto-git (master)
$ git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   app.txt

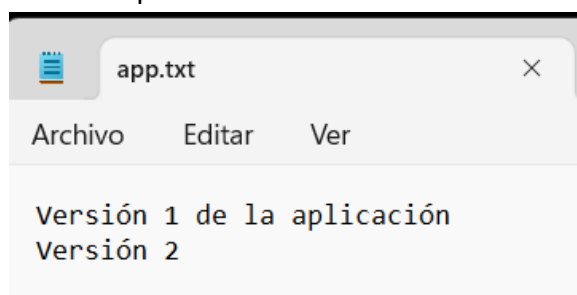
no changes added to commit (use "git add" and/or "git commit -a")
```

Ahora para descartar los cambios hacemos un checkout

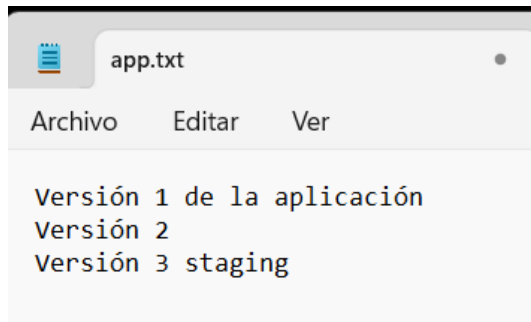
```
Alumno04@PC13 MINGW64 /c/git/proyecto-git (master)
$ git checkout app.txt
Updated 1 path from the index

Alumno04@PC13 MINGW64 /c/git/proyecto-git (master)
$ git status
On branch master
nothing to commit, working tree clean
```

Y vemos que se ha realizado correctamente



Lo volvemos a modificar



Y lo subimos al staging

```
Alumno04@PC13 MINGW64 /c/git/proyecto-git (master)
$ git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   app.txt

no changes added to commit (use "git add" and/or "git commit -a")

Alumno04@PC13 MINGW64 /c/git/proyecto-git (master)
$ git add .
```

Para sacarlo del staging hacemos un restore y vemos con el status que se ha realizado correctamente

```
Alumno04@PC13 MINGW64 /c/git/proyecto-git (master)
$ git restore --staged app.txt

Alumno04@PC13 MINGW64 /c/git/proyecto-git (master)
$ git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   app.txt
```

Ahora volvemos a realizar el add y un commit final

```
Alumno04@PC13 MINGW64 /c/git/proyecto-git (master)
$ git add .

Alumno04@PC13 MINGW64 /c/git/proyecto-git (master)
$ git commit -m "commit final"
[master 248910f] commit final
1 file changed, 2 insertions(+), 1 deletion(-)

Alumno04@PC13 MINGW64 /c/git/proyecto-git (master)
$ git tree
* 248910f885894327ac0dab65a91a8bc3a5bebb89 (HEAD -> master) commit final
* e4c3e43d58187ab0e5af3971d82bf7183e9d675b Segundo commit de la aplicación
* 3cd9aa70f9880f3705f582f62f11c83ca09137a1 Primer commit de la aplicación
```

Ejercicio 3 – Trabajo con ramas:

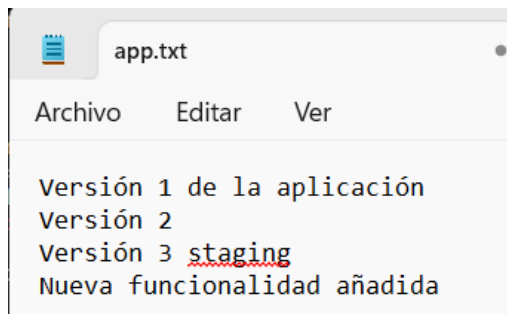
Para crear la rama usamos un branch y para cambiar usamos un switch

```
Alumno04@PC13 MINGW64 /c/git/proyecto-git (master)
$ git branch feature-nueva-funcionalidad

Alumno04@PC13 MINGW64 /c/git/proyecto-git (master)
$ git switch feature-nueva-funcionalidad
Switched to branch 'feature-nueva-funcionalidad'

Alumno04@PC13 MINGW64 /c/git/proyecto-git (feature-nueva-funcionalidad)
```

Modificamos el app.txt



```
app.txt
Archivo  Editar  Ver

Versión 1 de la aplicación
Versión 2
Versión 3 staging
Nueva funcionalidad añadida
```

Hacemos un commit

```
Alumno04@PC13 MINGW64 /c/git/proyecto-git (feature-nueva-funcionalidad)
$ git status
On branch feature-nueva-funcionalidad
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   app.txt

no changes added to commit (use "git add" and/or "git commit -a")

Alumno04@PC13 MINGW64 /c/git/proyecto-git (feature-nueva-funcionalidad)
$ git add .

Alumno04@PC13 MINGW64 /c/git/proyecto-git (feature-nueva-funcionalidad)
$ git commit -m "Primer commit feature"
[feature-nueva-funcionalidad f95dbfd] Primer commit feature
1 file changed, 2 insertions(+), 1 deletion(-)
```

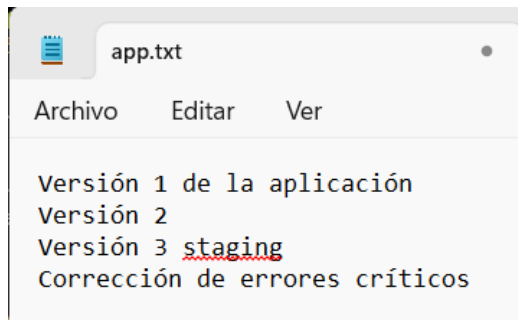
Cambiamos a master y vemos que los cambios no están

```
Alumno04@PC13 MINGW64 /c/git/proyecto-git (feature-nueva-funcionalidad)
$ git switch master
Switched to branch 'master'

Alumno04@PC13 MINGW64 /c/git/proyecto-git (master)
$ git tree
* 248910f885894327ac0dab65a91a8bc3a5bebb89 (HEAD -> master) commit final
* e4c3e43d58187ab0e5af3971d82bf7183e9d675b Segundo commit de la aplicación
* 3cd9aa70f9880f3705f582f62f11c83ca09137a1 Primer commit de la aplicación
```

Ejercicio 4 – Merge y conflictos:

Para empezar modificamos app.txt



Y hacemos commit

```
Alumno04@PC13 MINGW64 /c/git/proyecto-git (master)
$ git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   app.txt

no changes added to commit (use "git add" and/or "git commit -a")

Alumno04@PC13 MINGW64 /c/git/proyecto-git (master)
$ git add .

Alumno04@PC13 MINGW64 /c/git/proyecto-git (master)
$ git commit
Aborting commit due to empty commit message.

Alumno04@PC13 MINGW64 /c/git/proyecto-git (master)
$ git commit -m "Commit apartado merge"
[master 21f6ecb] Commit apartado merge
1 file changed, 2 insertions(+), 1 deletion(-)
```

Ahora fusionamos las ramas con un merge, para resolver el conflicto hacemos el checkout –ours y hacemos un add, después hacemos un merge –continue para seguir con el y vemos que se ha realizado correctamente con el tree

```
Alumno04@PC13 MINGW64 /c/git/proyecto-git (master)
$ git merge feature-nueva-funcionalidad
Auto-merging app.txt
CONFLICT (content): Merge conflict in app.txt
Automatic merge failed; fix conflicts and then commit the result.

Alumno04@PC13 MINGW64 /c/git/proyecto-git (master|MERGING)
$ git checkout --ours app.txt
Updated 1 path from the index

Alumno04@PC13 MINGW64 /c/git/proyecto-git (master|MERGING)
$ git add .

Alumno04@PC13 MINGW64 /c/git/proyecto-git (master|MERGING)
$ git merge --continue
[master e6d2f1e] Merge branch 'feature-nueva-funcionalidad'

Alumno04@PC13 MINGW64 /c/git/proyecto-git (master)
$ git tree
* e6d2f1e6902d3a5253a262047db309b0b1ded774 (HEAD -> master) Merge branch 'feature-nueva-funcionalidad'
|
| * f95dbfd70827380fd918b426698713ef0260d283 (feature-nueva-funcionalidad) Primer commit feature
| * | 21f6ecb5fe738523c02c90629738058155ac6b2d Commit apartado merge
|/
* 248910f885894327ac0dab65a91a8bc3a5bebb89 commit final
* e4c3e43d58187ab0e5af3971d82bf7183e9d675b Segundo commit de la aplicación
* 3cd9aa70f9880f3705f582f62f11c83ca09137a1 Primer commit de la aplicación
```

Ejercicio 5 – Cierre:

Para eliminar la rama hacemos un branch -d y vemos el historial con un log

```
Alumno04@PC13 MINGW64 /c/git/proyecto-git (master)
$ git branch -d feature-nueva-funcionalidad
Deleted branch feature-nueva-funcionalidad (was f95dbfd).

Alumno04@PC13 MINGW64 /c/git/proyecto-git (master)
$ git log
commit e6d2f1e6902d3a5253a262047db309b0b1ded774 (HEAD -> master)
Merge: 21f6ecb f95dbfd
Author: Javier Ruiz <javier.ruiz@salesianoslosboscos.com>
Date:   Fri Jan 9 09:57:50 2026 +0100

    Merge branch 'feature-nueva-funcionalidad'

commit 21f6ecb5fe738523c02c90629738058155ac6b2d
Author: Javier Ruiz <javier.ruiz@salesianoslosboscos.com>
Date:   Fri Jan 9 09:53:25 2026 +0100

    Commit apartado merge

commit f95dbfd70827380fd918b426698713ef0260d283
Author: Javier Ruiz <javier.ruiz@salesianoslosboscos.com>
Date:   Fri Jan 9 09:49:12 2026 +0100

    Primer commit feature

commit 248910f885894327ac0dab65a91a8bc3a5bebb89
Author: Javier Ruiz <javier.ruiz@salesianoslosboscos.com>
Date:   Fri Jan 9 09:45:06 2026 +0100

    commit final

commit e4c3e43d58187ab0e5af3971d82bf7183e9d675b
Author: Javier Ruiz <javier.ruiz@salesianoslosboscos.com>
Date:   Fri Jan 9 09:31:58 2026 +0100

    Segundo commit de la aplicación

commit 3cd9aa70f9880f3705f582f62f11c83ca09137a1
Author: Javier Ruiz <javier.ruiz@salesianoslosboscos.com>
Date:   Fri Jan 9 09:29:05 2026 +0100

    Primer commit de la aplicación
```


Responde brevemente:

o ¿Para qué sirven las ramas?

Sirven para modificar y trabajar en un proyecto sin afectar al trabajo principal.

o ¿Qué ha causado el conflicto?

Que en una misma rama teníamos varias versiones, para solucionarlo solamente había que elegir una de las versiones.

o ¿Qué ventajas ofrece Git frente a no usar control de versiones?

- Tiene un historial de versiones para saber quién, cuándo y qué se ha modificado.
- Ramas que permiten trabajar sin miedo a cambiar el proyecto principal.
- Un trabajo en equipo limpio al poder trabajar sin solaparse y poder juntar ideas.